



Documentación de proyecto

Web de Atenia: animación del logo y Pong contra la IA

Autor **Mario Castro**
Fecha **5 de octubre de 2026**
Versión **1.0**



Índice

1 Resumen	3
1.1 Recorrido de la web	3
1.2 Qué representa el logo	3
2 Funcionamiento de la web	4
2.1 Intro: la animación del logo	4
2.2 Transformación en el juego	5
2.3 Inicio de la partida	6
2.4 Partida	6
2.5 Final	7
2.6 Móvil	7
3 Puesta en marcha	9
3.1 Requisitos	9
3.2 Ejecutar en local	9
3.3 Publicar	9
4 Manual de uso del código	10
4.1 Estructura	10
4.2 Textos	10
4.3 Colores	10
4.4 Tiempos de la animación	11
4.5 Dificultad	11
4.6 Sustituir el logo	11
5 Arquitectura	12
5.1 Visión general	12
5.2 Origen del logo	12
5.3 Línea de tiempo de la intro	12
5.4 Transformación logo → Pong	12
5.5 Juego	13
5.6 Alojamiento	13
6 Pruebas	14
7 Limitaciones conocidas	15
8 Plantilla para otros documentos	16

1 Resumen

Atenia Healthcare Robotics desarrolla un robot para hospitales con un sistema de seguimiento de calorías. Esta web es su carta de presentación: enseña la marca con una animación del logo y después invita a jugar un Pong contra la IA en el que las piezas del propio logo se convierten en el juego.

En producción

atenia-ingenia.web.app
Firebase Hosting · proyecto
[atenia-ingenia](#)

Código

Repositorio [atenia-web](#)
HTML, CSS y JavaScript, sin
dependencias ni compilación

1.1 Recorrido de la web

1. **Intro.** El robot se monta pieza a pieza, el nombre sale de detrás de él y sus ojos escanean con un láser verde que escribe "Healthcare Robotics".
2. **Jugar.** Al terminar la animación aparece el botón **Jugar**.
3. **Transformación.** "Aten" se funde en tu pala, la "a" en la de la IA, y las piezas del robot viajan hasta encajar en la pelota.
4. **Inicio de partida.** Una tarjeta explica las reglas, pide tu nombre y muestra los controles.
5. **Partida.** Gana quien llegue antes a 5 puntos. Un marcador con tu nombre sigue el resultado.
6. **Final.** Se muestra el resultado y puedes pedir **Revancha** o volver al logo.

1.2 Qué representa el logo

El robot hace de "i" de *Atenia*. La cruz médica del cuerpo remite al hospital. El láser de los ojos escanea como lo haría el robot con una bandeja de comida para contar sus calorías, y ese escaneo es el que escribe el eslogan. En la web, los ojos del logo original (naranjas) pasan a verde, el mismo color del láser.

Atenia
Healthcare Robotics

2 Funcionamiento de la web

Las capturas de este capítulo se tomaron de la web en ejecución, en un navegador Chrome sin ventana a 1280 × 720 píxeles (y a 390 × 844 para móvil). Los fotogramas de la intro se congelaron en el instante exacto indicado en cada pie.

2.1 Intro: la animación del logo

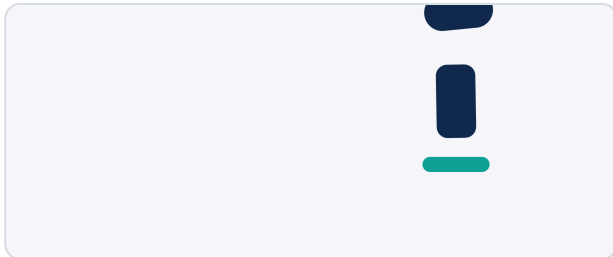


Figura 1 · 0,55 s: la base, el cuerpo y la cabeza llegan desde fuera de la pantalla.

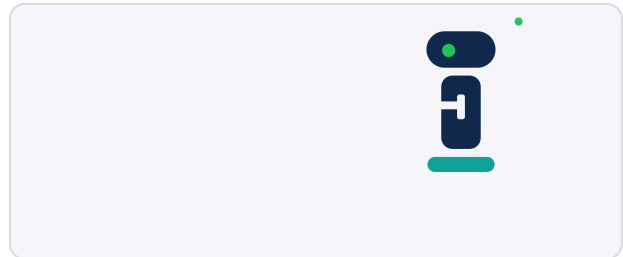


Figura 2 · 1,15 s: la cruz ya está formada y los ojos verdes se encienden.



Figura 3 · 1,75 s: las letras salen de detrás del robot.



Figura 4 · 3,30 s: el láser escanea y va escribiendo el eslogan.

Al acabar el escaneo, la cruz late dos veces, un brillo recorre la base, el robot parpadea y saluda con la cabeza. A los 5,6 s aparece el botón **Jugar**.



Figura 5 · Final de la intro con el botón Jugar.

2.2 Transformación en el juego

Al pulsar **Jugar**, el logo se desmonta. Las letras "A", "t", "e" y "n" vuelan hacia la izquierda y se funden en tu pala; la "a" hace lo mismo con la pala de la IA. A la vez, las siete piezas del robot (cabeza, dos ojos, cuerpo, dos barras de la cruz y base) viajan hasta el centro y se encajan en una placa blanca: la pelota es el icono del robot.

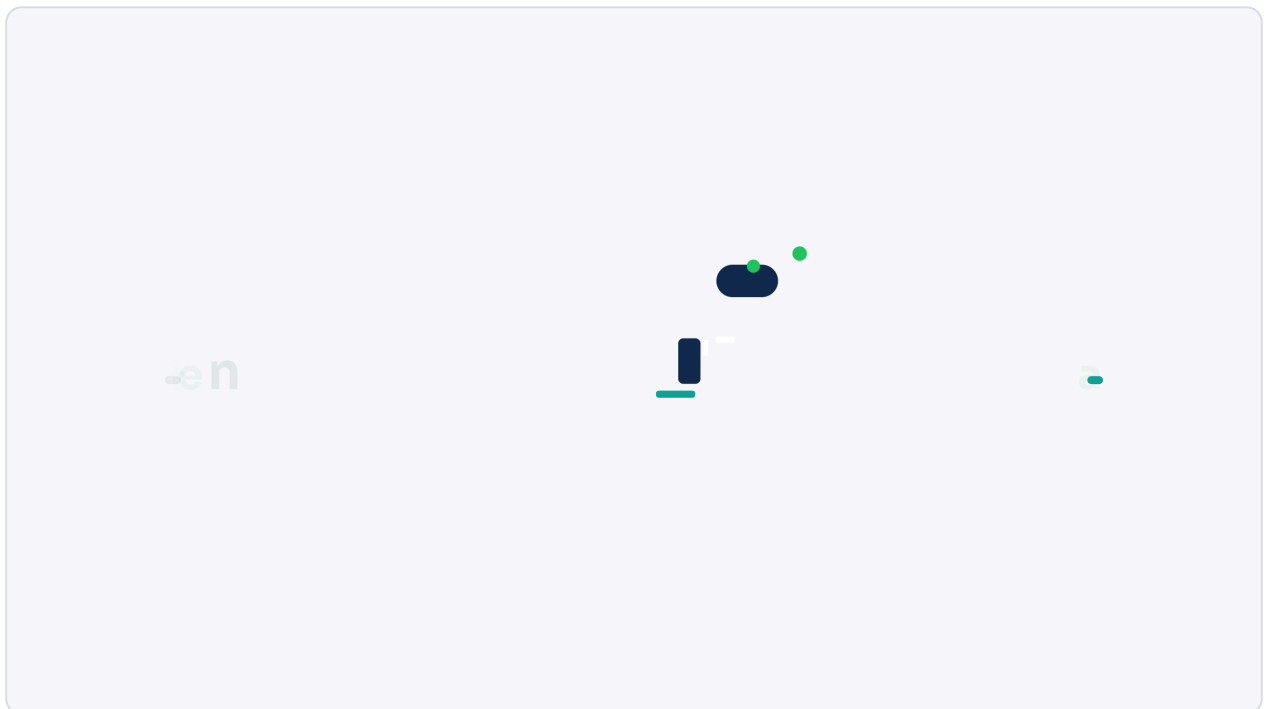


Figura 6 · A mitad de la transformación (0,7 s): las piezas viajan hacia la pelota y las letras hacia las palas.

2.3 Inicio de la partida

Antes de jugar aparece una tarjeta con las reglas (“el primero que llegue a 5 puntos gana”), un campo para tu nombre y los controles. El nombre se guarda en el navegador, así que la próxima vez ya aparece escrito.

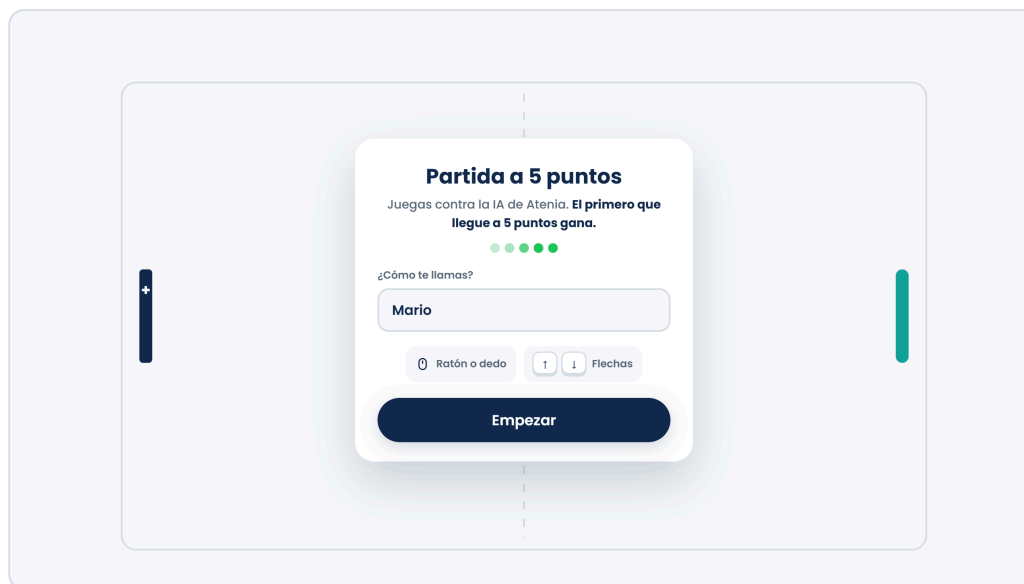


Figura 7 · Tarjeta de inicio con el nombre escrito.

2.4 Partida

Tu pala es el cuerpo del robot (azul marino, con la cruz) y la de la IA es la base (teal). La pelota gira en la dirección en la que se mueve y brilla en verde cada vez que la golpea una pala. Arriba, el marcador muestra tu inicial, tu nombre, los puntos con cinco indicadores y el de la IA (“Atenia IA”).

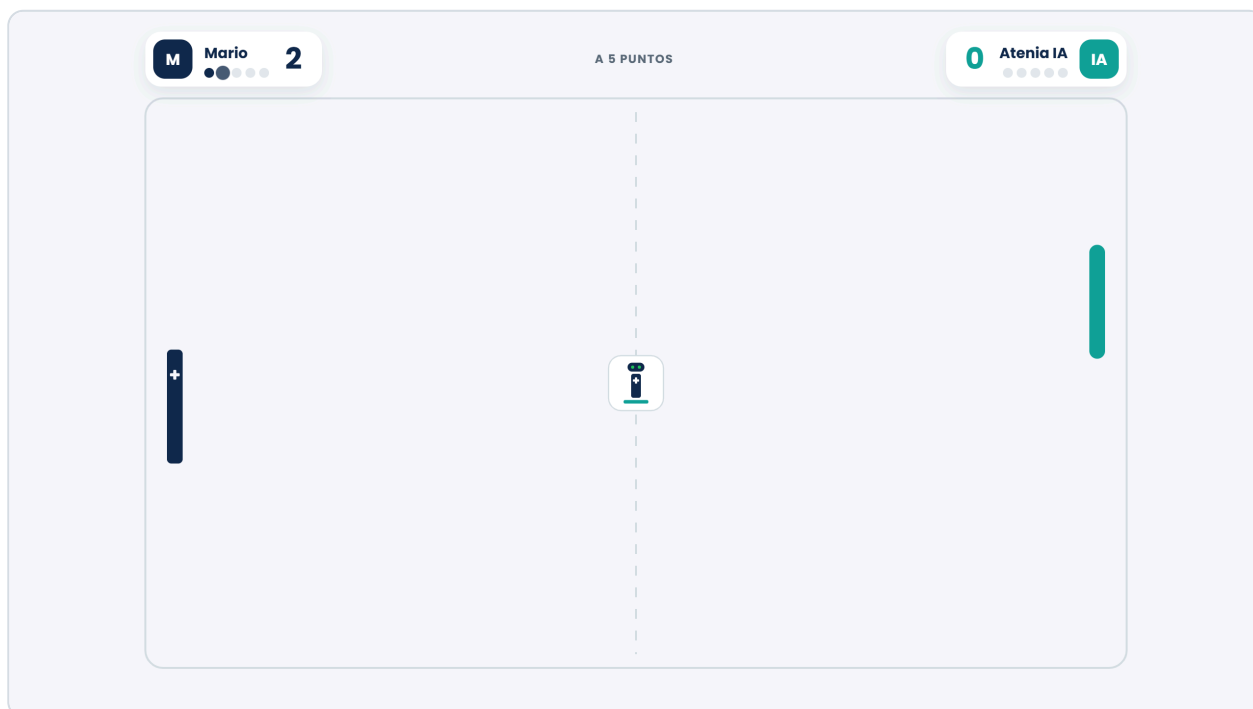


Figura 8 · Partida en curso: 2 – 0 a favor del jugador.

2.5 Final

Cuando alguien llega a 5 puntos, el juego se detiene y aparece el resultado: “¡Has ganado, *nombre!*” o “Gana la IA”. **Revancha** empieza otra partida directamente y **Ver el logo** recarga la página con la intro.

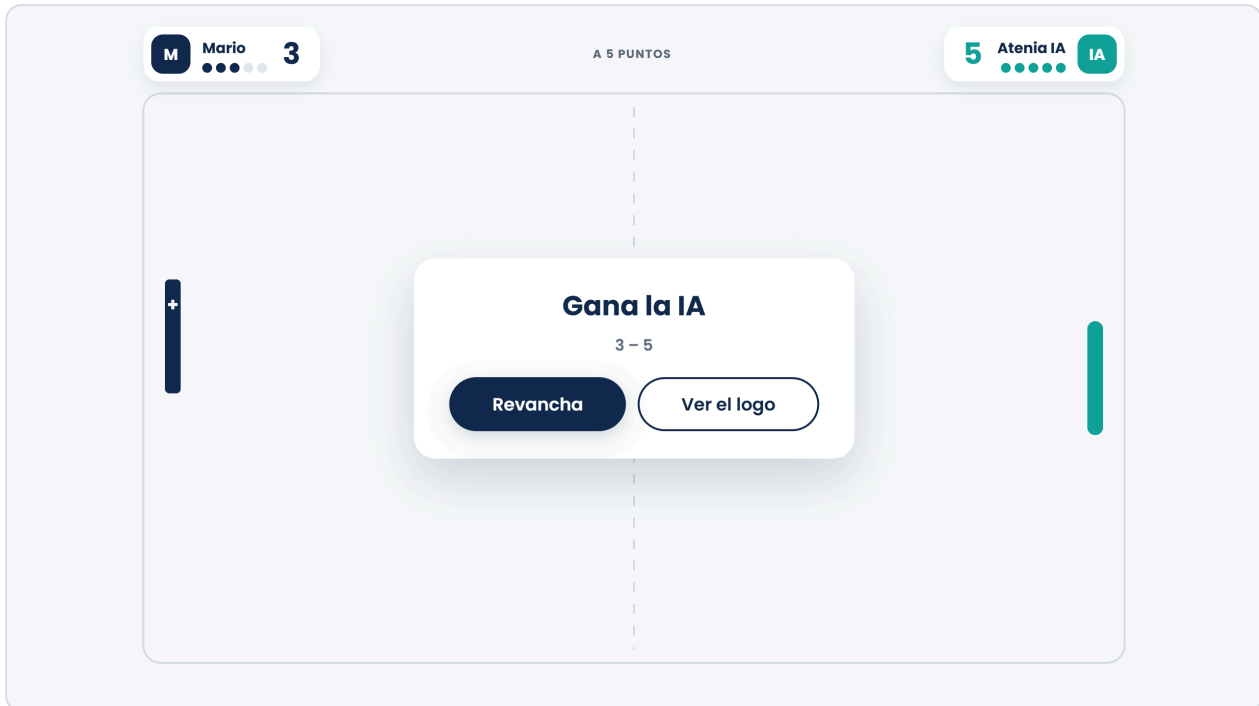


Figura 9 · Pantalla final de una partida real que ganó la IA por 3 – 5.

2.6 Móvil

En vertical, el campo es más alto que ancho para aprovechar la pantalla, y la pala se mueve deslizando el dedo. Bajo el campo aparece un recordatorio de los controles al empezar.



Figura 10 · Intro en un móvil (390 × 844).

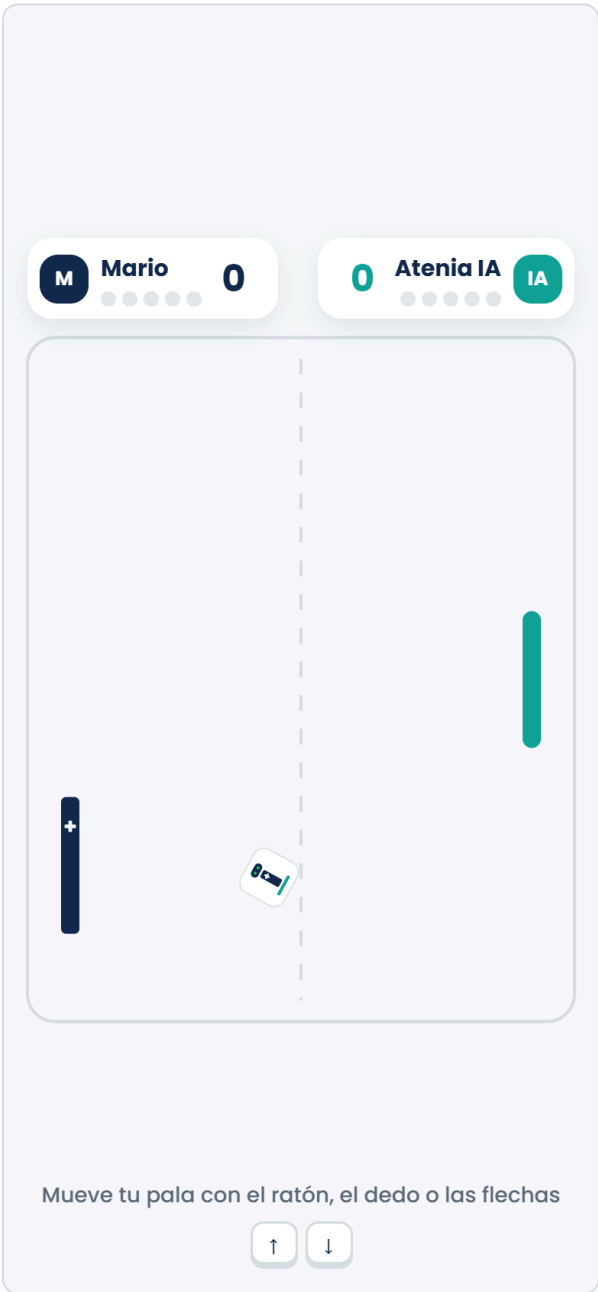


Figura 11 · Partida en el móvil.

3 Puesta en marcha

3.1 Requisitos

- [Node.js](#).
- Firebase CLI: `npm i -g firebase-tools`.

No hay que instalar dependencias ni compilar: la carpeta `public/` es exactamente lo que se publica.

3.2 Ejecutar en local

```
cd atenia-web
firebase emulators:start --only hosting
```

La web queda en `http://localhost:5050`, servida por el emulador de Firebase Hosting igual que en producción. `Ctrl+C` lo detiene. Como alternativa vale cualquier servidor estático, por ejemplo `npx serve public`. No conviene abrir `index.html` con doble clic: en `file://` algunos navegadores bloquean partes de la página.

3.3 Publicar

```
firebase login           # solo la primera vez en cada ordenador
firebase deploy --only hosting
```

La web queda en atenia-ingenia.web.app y en atenia-ingenia.firebaseio.com.

Tarea	Cómo
Publicar en otro proyecto	<code>firebase deploy --only hosting --project <id></code>
Volver a una versión anterior	Consola de Firebase → Hosting → Historial de versiones → Restaurar
Dominio propio	Consola de Firebase → Hosting → Agregar dominio personalizado

Sin caché del navegador

`firebase.json` envía `Cache-Control: no-cache` en todos los archivos. El navegador comprueba siempre si hay una versión nueva, así que cada despliegue se ve al momento, sin recargar a la fuerza.

4 Manual de uso del código

4.1 Estructura

```

atenia-web/
├── public/
│   ├── index.html  página: estilos, SVG animado del logo, botón, canvas,
│   │               tarjeta de inicio, marcador y diálogo final
│   └── game.js      transformación logo → Pong, juego, IA y controles
├── docs/
│   ├── MANUAL.md,  ARQUITECTURA.md
│   └── typst/       este documento y la plantilla
├── firebase.json    Hosting y emulador (puerto 5050)
└── .firebaserc      proyecto por defecto: atenia-ingenia
  
```

4.2 Textos

Toda la interfaz está en español.

Texto	Dónde
Título y descripción de la pestaña	index.html: <code><title></code> y <code><meta name="description"></code>
Botón Jugar	index.html: <code><button id="play"></code>
Tarjeta de inicio	index.html: <code><form id="start"></code>
Marcador	index.html: <code><div id="hud"></code> ("A 5 PUNTOS", "Atenia IA")
Ayuda de controles	index.html: <code><p id="hint"></code>
Botones finales	index.html: <code>#again</code> (Revancha) y <code>#home</code> (Ver el logo)
Resultado final	game.js: función <code>point()</code>

4.3 Colores

La paleta de la marca, sin el naranja del logo original:

				
Azul marino #13294b	Teal #12a39a	Verde #22c55e	Gris #5c6b7a	Fondo #f7f9fb

Están definidos en tres sitios, y hay que cambiarlos en todos:

- `game.js`, al principio: `NAVY`, `TEAL`, `GREEN`, `BG`, `GREY`. Los usa el juego.
- `index.html`, en `:root`: `--navy`, `--teal`, `--green`, `--bg`, `--grey`. Los usan la página, la tarjeta y el marcador.
- El SVG del logo lleva el color escrito en cada pieza. El verde tiene además dos tonos claros: `#C9F7DA` (brillo de los ojos) y `#E3FCEC` (línea de escaneo).

4.4 Tiempos de la animación

La animación combina animaciones CSS (`animation: nombre duración retardo ...`) para mover piezas y animaciones SMIL (`<animate begin="..." dur="...">`) para el láser. La tabla completa está en el capítulo de arquitectura. Si cambia la duración total, hay que ajustar también `INTRO_MS` en `game.js` , que decide cuándo aparece el botón **Jugar** (5600 ms).

4.5 Dificultad

Ajuste	Dónde (<code>game.js</code>)	Valor actual
Puntos para ganar	<code>WIN_SCORE</code>	5
Velocidad de saque	<code>serve() : field.w * 0.55</code>	0,55 anchos de campo/s
Aceleración por golpe	<code>hitPaddle() : * 1.06</code>	+6 % por golpe
Velocidad máxima	<code>hitPaddle() : field.w * 1.4</code>	1,4 anchos/s
Velocidad de la IA	<code>step() : field.h * 1.05</code>	1,05 altos de campo/s
Error de la IA	<code>hitPaddle() : p.h * 0.9</code>	hasta ±45 % de la pala
Tamaño de las palas	<code>layout() : fh * 0.2</code>	20 % del alto del campo
Tamaño de la pelota	<code>layout() : ball.r</code>	2,8 % del ancho del campo

Para una IA más difícil, sube su velocidad o baja su error; para una más fácil, al revés. Si cambias `WIN_SCORE` , cambia también los textos “5 puntos” de la tarjeta de inicio y del marcador.

4.6 Sustituir el logo

El juego encuentra las piezas del logo por su `id` . Si se cambia el logo hay que conservarlos (o actualizarlos en `game.js`):

id	Pieza
<code>LA , Lt , Le , Ln , La</code>	Letras A, t, e, n, a
<code>tagline</code>	Grupo con “Healthcare Robotics”
<code>head</code>	Cabeza; contiene <code>eyeL</code> y <code>eyeR</code>
<code>body</code>	Cuerpo del robot
<code>cross</code>	Cruz; contiene <code>cv</code> (vertical) y <code>ch</code> (horizontal)
<code>base</code>	Barra teal
<code>shine</code>	Brillo que recorre la base

La pelota no lee el SVG: dibuja el robot a partir de la constante `ROBOT` de `game.js` , con las coordenadas del icono en una placa de 140 × 140. Si cambia la forma del robot, hay que actualizarla también.

5 Arquitectura

5.1 Visión general

Es una sola página estática con dos estados. En la **intro**, un `<svg id="logo">` escrito dentro del HTML reproduce la animación. En el **juego**, un `<canvas>` a pantalla completa dibuja la transformación y el Pong, con la tarjeta de inicio, el marcador y el diálogo final como HTML por encima. El SVG va dentro del HTML, y no como `<img>`, porque `game.js` necesita leer la posición de cada pieza en pantalla (`getBoundingClientRect`) y animar las letras durante la transformación.

5.2 Origen del logo

El logo sale de la hoja `atenia-robotics-logo.svg`, que tiene tres variantes: logotipo claro, logotipo oscuro e icono de app. La web usa el **logotipo claro** para la intro y el **icono** para la pelota. Las piezas son los vectores originales, sin redibujar.

5.3 Línea de tiempo de la intro

Tiempo	Qué pasa	Mecanismo
0,00 s	La base teal entra desde la izquierda	CSS <code>#base</code>
0,15 s	El cuerpo sube desde abajo girando	CSS <code>#body</code>
0,35 s	La cabeza cae desde arriba	CSS <code>#head</code>
0,65 – 0,75 s	Las barras de la cruz se cruzan	CSS <code>#cv</code> , <code>#ch</code>
1,00 – 1,08 s	Los ojos se encienden	CSS <code>#eyeL</code> , <code>#eyeR</code>
1,40 – 1,70 s	n, e, t, A salen a la izquierda; la "a" a la derecha	CSS <code>slide</code>
2,45 s	Los ojos se cargan con un brillo verde	SMIL <code>#glowL</code> , <code>#glowR</code>
2,75 s	Aparecen los haces láser	SMIL <code>#lasers</code>
2,85 – 3,85 s	El escaneo revela el eslogan	SMIL <code>#scanClip</code>
4,10 s	La cruz late y un brillo recorre la base	CSS <code>#cross</code> , <code>#shine</code>
4,60 s	El robot parpadea	CSS <code>blink</code>
4,90 s	El robot saluda con la cabeza	CSS <code>nod</code>
5,60 s	Aparece el botón Jugar	INTRO_MS

Por qué dos sistemas. CSS mueve piezas enteras con `transform`. El láser necesita animar geometría (los puntos del haz, las coordenadas de las líneas y el ancho del recorte), y eso solo lo permite SMIL. Lo que debe ir sincronizado con el láser también usa SMIL, para depender del mismo reloj.

Las piezas usan `transform-box: fill-box` para girar sobre su propio centro y `fill-mode: both` para estar fuera de la pantalla antes de empezar. Las animaciones secundarias (`nod`, `blink`, `beat`) usan `forwards`: con `both` pisarían la entrada desde el primer instante. Con `prefers-reduced-motion` todo se reduce a casi cero.

5.4 Transformación logo → Pong

1. `layout()` calcula el campo y las posiciones finales de palas y pelota.

2. Se lee el rectángulo en pantalla de cada pieza del robot y se ocultan las originales del SVG; a partir de ahí las dibuja el canvas, que está encima.
3. Cada pieza tiene su destino dentro de la pelota según la constante `ROBOT`, escalada al radio de la pelota.
4. Las letras viajan a las palas con la Web Animations API (`e1.animate`), que tiene prioridad sobre las animaciones CSS.
5. `morph()` interpola las piezas durante 1,2 s, con un pequeño escalonado, mientras las palas crecen, aparece el campo y la placa blanca se forma detrás de las piezas.
6. Al terminar se oculta la intro y se muestra la tarjeta de inicio. La partida empieza al enviar el formulario.

5.5 Juego

Estado	Contenido
<code>field</code>	Rectángulo del campo, en píxeles CSS
<code>P, AI</code>	Palas: <code>{x, y, w, h}</code>
<code>ball</code>	<code>{x, y, vx, vy, r, spin}</code>
<code>score</code>	<code>{p, ai}</code>
<code>playerName</code>	Nombre del jugador; se guarda en <code>localStorage (ateniaName)</code> , máximo 14 caracteres

- **Bucle.** `loop()` corre con `requestAnimationFrame` y llama a `step(dt)` y `draw()`. El tiempo sale de `performance.now()` y `dt` se limita a 33 ms para que la pelota no atraviese las palas tras una pausa.
- **Física.** Saque desde el centro con un ángulo aleatorio de $\pm 17^\circ$ y 0,8 s de espera. El ángulo de salida depende de dónde golpea la pelota en la pala, hasta unos 55° . La velocidad sube un 6 % por golpe hasta 1,4 anchos de campo por segundo.
- **IA.** Cuando la pelota va hacia ella, persigue `ball.y + aiErr`; si no, vuelve al centro. Su velocidad está limitada y `aiErr` cambia al azar en cada golpe, por eso se le puede ganar.
- **Controles.** Ratón y dedo fijan `pointerY` y la pala lo sigue con suavizado. El teclado (`↑ ↓, W S`) la mueve a velocidad fija. Mientras escribes el nombre, las teclas no mueven la pala.
- **Marcador.** Es HTML (`#hud`) colocado justo encima del campo. `renderHud()` actualiza nombre, inicial, puntos e indicadores, con un pequeño “pop” en el punto nuevo.
- **Tamaño adaptable.** Campo de hasta 1000 px de ancho, con proporción 0,62 en horizontal y 1,25 en vertical. Al cambiar el tamaño de la ventana se reescala todo para que la partida siga donde estaba. El canvas usa `devicePixelRatio` para verse nítido.

5.6 Alojamiento

Clave de <code>firebase.json</code>	Función
<code>hosting.public</code>	Carpeta publicada: <code>public/</code>
<code>hosting.headers</code>	Cache-Control: no-cache en todos los archivos
<code>emulators.hosting.port</code>	5050 para desarrollo local; la interfaz del emulador está desactivada
<code>.firebaserc (aparte)</code>	Proyecto por defecto: <code>atenia-ingenia</code>

6 Pruebas

No hay tests automáticos en el repositorio. Antes de publicar conviene repasar:

Comprobación	Resultado esperado
Intro	La animación dura unos 5,7 s y luego aparece Jugar
Transformación	Letras a las palas, piezas del robot a la pelota
Inicio	La tarjeta pide el nombre y lo recuerda al volver
Partida	El marcador suma bien y la pelota acelera
Final	Resultado correcto; Revancha y Ver el logo funcionan
Móvil	Campo vertical y control deslizando el dedo
Consola (F12)	Sin errores

Desde la consola del navegador se puede inspeccionar el juego con `__atenia.state()`, que devuelve `running`, `score`, `ball`, `P`, `AI` y `field`.

Pestañas en segundo plano

Si la pestaña no está visible, el navegador pausa `requestAnimationFrame` y el juego se congela hasta volver a ella. Es el comportamiento normal de los navegadores, no un fallo.

Para este documento, la web se probó en Chrome sin ventana controlado por script: una partida completa sin mover la pala (gana la IA 0 – 5), partidas con un “piloto automático” que sigue la pelota, y la vista de móvil. No hubo errores en la consola.

7 Limitaciones conocidas

- **Pestaña en segundo plano:** la partida se congela hasta volver (ver Pruebas).
- **Choque de la pelota:** se calcula como un círculo, así que al girar las esquinas de la placa pueden solaparse un poco con la pala.
- **Colores repetidos:** la paleta está en el CSS, en `game.js` y en el SVG.
- **Sin tests automáticos:** la verificación es manual.

8 Plantilla para otros documentos

Este documento usa `plantilla-atenia.typ`, que está en la misma carpeta y sirve para cualquier otro documento de Atenia. Incluye la portada, la cabecera con el logotipo completo en la esquina, el pie con la paginación, la fuente Poppins y la paleta sin naranja.

```
#import "plantilla-atenia.typ": *

#show: documento.with(
  titulo: "Informe de pruebas",
  subtítulo: "Robot Atenia · hospital piloto",
  autor: "Mario Castro",
  version: "0.1",
  // portada: false, indice: false  para documentos cortos
)

= Introducción
Texto normal.

#nota(titulo: "Importante")[Recuadro destacado.]
#captura("capturas/foto.png")[Pie de la figura.]
```

Componente	Uso
<code>documento.with(...)</code>	Estilo completo: <code>titulo</code> , <code>subtitulo</code> , <code>autor</code> , <code>fecha</code> , <code>version</code> , <code>portada</code> , <code>indice</code>
<code>nota(tipo, titulo)[...]</code>	Recuadro: "info" (teal), "ok" (verde) o "aviso" (azul)
<code>captura(ruta)[pie]</code>	Captura con marco redondeado y pie de figura; <code>ancho</code> : opcional
<code>paleta((nombre, color), ...)</code>	Muestras de color con su código
<code>chip[texto]</code>	Etiqueta pequeña de color
<code>logo()</code> , <code>icono()</code>	Logotipo completo e icono del robot
<code>azul</code> , <code>teal</code> , <code>verde</code> , <code>gris</code> , <code>fondo</code> , <code>borde</code>	Colores de la marca

Para compilar, con la fuente incluida en la carpeta `fonts/`:

```
typst compile --font-path fonts documentacion-proyecto.typ
```

Hay un punto de partida listo para copiar en `plantilla-ejemplo.typ`.